

Turing Interview Prep: Senior Software Engineer – Go 1.26+ (LLM Evaluation & Repository Validation)

Updated for Go 1.26 (February 2026) - Modern patterns, errgroup, new features

Complete preparation guide for Golang-focused interview and coding rounds using the latest Go 1.26 features and best practices.

Role Overview

Focus Areas:

1. **Go/Golang Expertise** - Modern concurrency patterns (errgroup), channels, interfaces, performance
 2. **LLM Evaluation** - Metrics, benchmarking, quality assessment
 3. **Repository Validation** - Code analysis, static checking, CI/CD
 4. **System Design** - Scalable evaluation pipelines
-

What's New in Go 1.26

Key Features You Must Know

1. **new(expr)** - Allocate and initialize pointers inline
 2. **Green Tea GC** - New garbage collector enabled by default (pacer-v3)
 3. **Goroutine leak detector** - Built-in leak detection in tests
 4. **New packages:** `crypto/hpke`, `crypto/mlkem/mlkemtest`, `testing/cryptotest`
 5. **errors.Append()** - Alternative to errors.Join() for collecting errors
 6. **Performance improvements** - 5-15% faster compilation, reduced memory usage
-

Go 1.26 Modern Concurrency Patterns

1. Structured Concurrency with errgroup (MODERN WAY)

✗ OLD STYLE (Don't use anymore):

```
var wg sync.WaitGroup
for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        // work
    }(i)
}
wg.Wait()
```

☑ MODERN GO 1.26 STYLE:

```
package main

import (
    "context"
    "fmt"
    "time"

    "golang.org/x/sync/errgroup"
)

// Modern concurrent LLM evaluation with errgroup
func EvaluateConcurrently(ctx context.Context, tasks []EvaluationTask)
([]EvaluationResult, error) {
    g, ctx := errgroup.WithContext(ctx)

    // Set concurrency limit (Go 1.20+ feature)
    g.SetLimit(10) // Max 10 concurrent goroutines

    results := make([]EvaluationResult, len(tasks))

    // Launch goroutines - errgroup handles all synchronization
    for i, task := range tasks {
        // Go 1.22+ automatically captures loop variables correctly
        g.Go(func() error {
            result, err := evaluateTask(ctx, task)
            if err != nil {
                return fmt.Errorf("task %d failed: %w", i, err)
            }
            results[i] = result
            return nil
        })
    }

    // Wait for all goroutines and collect first error
    if err := g.Wait(); err != nil {
        return nil, err
    }

    return results, nil
}

type EvaluationTask struct {
    ModelName string
    Prompt    string
    Response  string
}

type EvaluationResult struct {
    Task      EvaluationTask
    Score     float64
    Latency   time.Duration
    Timestamp time.Time
}
```

```

}

func evaluateTask(ctx context.Context, task EvaluationTask) (EvaluationResult,
error) {
    start := time.Now()

    select {
    case <-ctx.Done():
        return EvaluationResult{}, ctx.Err()
    case <-time.After(100 * time.Millisecond): // Simulate work
        score := calculateScore(task.Response)
        return EvaluationResult{
            Task:      task,
            Score:      score,
            Latency:    time.Since(start),
            Timestamp: time.Now(),
        }, nil
    }
}

func calculateScore(response string) float64 {
    // Placeholder scoring logic
    return 0.85
}

```

2. Modern Worker Pool with errgroup

☑ GO 1.26 PATTERN:

```

type WorkerPool struct {
    maxWorkers int
    semaphore  chan struct{}
}

func NewWorkerPool(maxWorkers int) *WorkerPool {
    return &WorkerPool{
        maxWorkers: maxWorkers,
        semaphore:  make(chan struct{}, maxWorkers),
    }
}

// Process tasks with bounded concurrency using errgroup
func (p *WorkerPool) Process(ctx context.Context, tasks []EvaluationTask)
([]EvaluationResult, error) {
    g, ctx := errgroup.WithContext(ctx)
    g.SetLimit(p.maxWorkers)

    results := make([]EvaluationResult, len(tasks))

    for i, task := range tasks {
        g.Go(func() error {

```

```

        result, err := evaluateTask(ctx, task)
        if err != nil {
            return err
        }
        results[i] = result
        return nil
    })
}

if err := g.Wait(); err != nil {
    return nil, err
}

return results, nil
}

```

3. Go 1.26 new(expr) Feature

☒ NEW IN GO 1.26:

```

type Config struct {
    Timeout      *time.Duration
    MaxRetries    *int
    EnableCache  *bool
}

// OLD WAY - verbose
func oldWay() Config {
    timeout := 30 * time.Second
    retries := 3
    cache := true

    return Config{
        Timeout:      &timeout,
        MaxRetries:    &retries,
        EnableCache:   &cache,
    }
}

// NEW GO 1.26 WAY - clean and inline
func modernWay() Config {
    return Config{
        Timeout:      new(30 * time.Second), // ☒ Direct expression!
        MaxRetries:    new(3),
        EnableCache:   new(true),
    }
}

```

4. Error Collection with errors.Append (Go 1.26)

☑ GO 1.26 PATTERN:

```
import (
    "errors"
    "fmt"
)

// Collect multiple errors without stopping
func ValidateModels(models []string) error {
    var errs error

    for _, model := range models {
        if err := validateModel(model); err != nil {
            errs = errors.Append(errs, fmt.Errorf("model %s: %w", model, err))
        }
    }

    return errs
}

func validateModel(model string) error {
    if model == "" {
        return errors.New("empty model name")
    }
    return nil
}

// Alternative: errors.Join for slice of errors
func ValidateWithJoin(models []string) error {
    errs := make([]error, 0)

    for _, model := range models {
        if err := validateModel(model); err != nil {
            errs = append(errs, fmt.Errorf("model %s: %w", model, err))
        }
    }

    return errors.Join(errs...)
}
```

5. Context-Aware Pipeline Pattern**☑ MODERN PIPELINE:**

```
// Modern pipeline with context and errgroup
func EvaluationPipeline(ctx context.Context, tasks []EvaluationTask) (<-chan
EvaluationResult, error) {
    g, ctx := errgroup.WithContext(ctx)
```

